

Analisi e Progettazione del Software

Altri pattern GRASP

Oliviero Riganelli

Pattern GRASP

- *Information Expert*
- *Creator*
- *High Cohesion*
- *Low Coupling*
- *Controller*
- *Polymorphism*
- *Indirection*
- *Pure Fabrication*
- *Protected Variations*

Pure Fabrication

- **Problema:** Quale oggetto deve avere la responsabilità quando non si vogliono violare High Cohesion e Low Coupling, o altri obiettivi, ma le soluzioni suggerite da Expert (ad esempio) non sono appropriate?
- **Soluzione:** assegnare un insieme di responsabilità altamente coeso a una classe artificiale o di convenienza, che non rappresenta un concetto del dominio del problema, ma piuttosto è una classe inventata per sostenere coesione alta, accoppiamento basso e riuso

Pure Fabrication

Vantaggi:

- Sostiene high cohesion
- Potenzialità di riuso aumentata

Svantaggi:

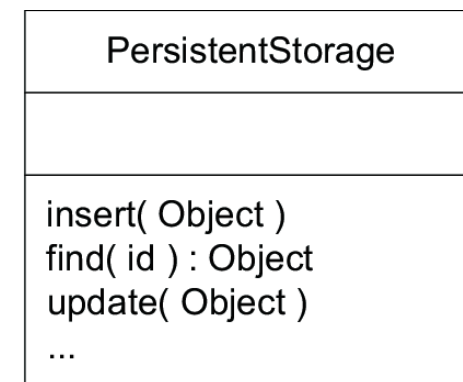
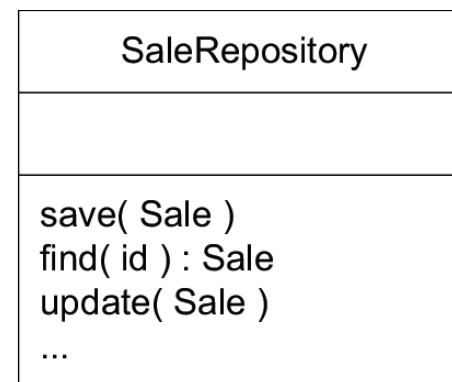
- Un utilizzo eccessivo può portare ad una decomposizione del codice: ovvero si potrebbe avere un eccesso di oggetti comportamentali che non possiedono le informazioni richieste per soddisfarli

Esempio POS NextGen

Vogliamo salvare l'oggetto di tipo Sale in una base di dati razionale. Solitamente questa responsabilità è assegnata alla classe stessa, ma Sale perderebbe di prestazioni:

- Meno coesione perché introduciamo le operazioni orientate ai db
- Aumento dell'accoppiamento all'interfaccia del db
- Diminuirebbe il riuso futuro della classe stessa
- Possibile codice duplicato con altre classi

Per Pure Fabrication



Errori comuni

- è un errore assegnare un insieme di responsabilità a una classe artificiale senza però sostenere coesione alta, accoppiamento basso e riuso
- è un errore non mostrare come una Pure Fabrication soddisfa le responsabilità che gli sono state assegnate (pensando che le Pure Fabrication siano “magiche” oppure sappiano progettarsi da sole)
 - *ovvero, quando assegniamo una responsabilità a una Pure Fabrication, è necessario progettare un’opportuna interazione , coordinata dalla Pure Fabrication*

Polymorphism

- **Problema:** come gestire alternative basate sul tipo? Come creare componenti software inseribili?
- **Soluzione:** quando le alternative e comportamenti correlati variano con il tipo, allora assegna la responsabilità del comportamento ai tipi per i quali il comportamento varia, utilizzando operazioni polimorfe

Polymorphism

Vantaggi:

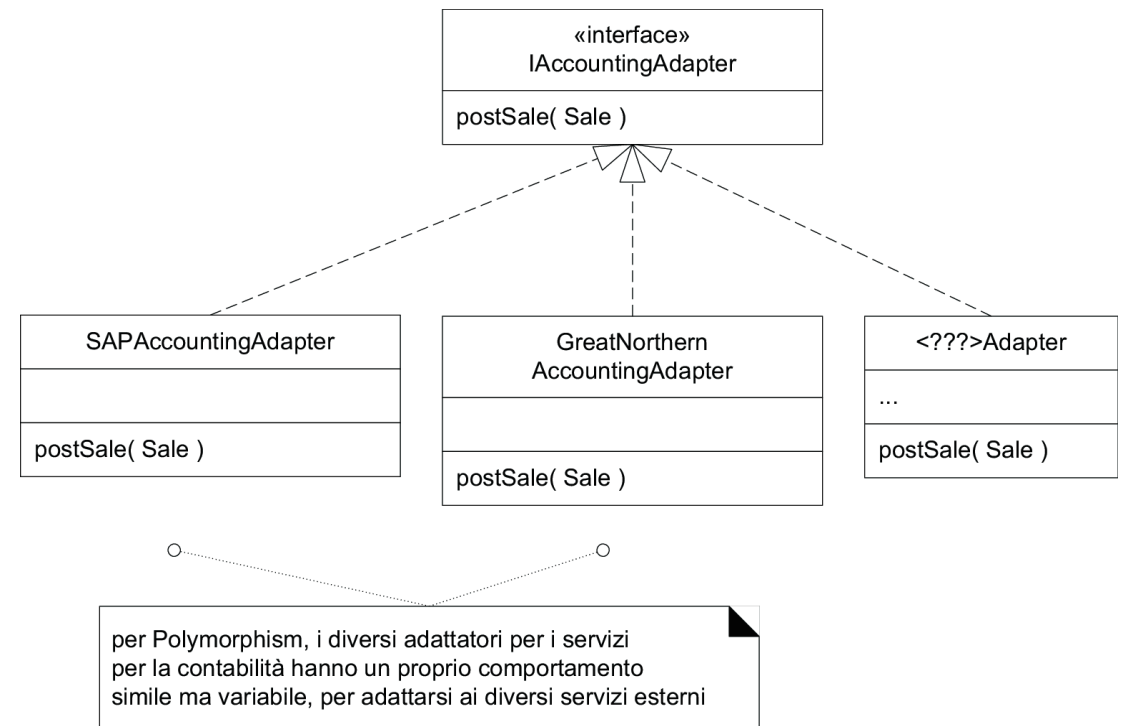
- le estensioni sono facili da aggiungere
- si possono introdurre nuove implementazioni senza influire sui client

Svantaggi:

- Progettare interfacce e polimorfismi per una “adattabilità” speculativa rispetto a delle possibili variazioni sconosciute può portare alla progettazione di variazioni di fatto improbabili che in realtà non si verificheranno mai

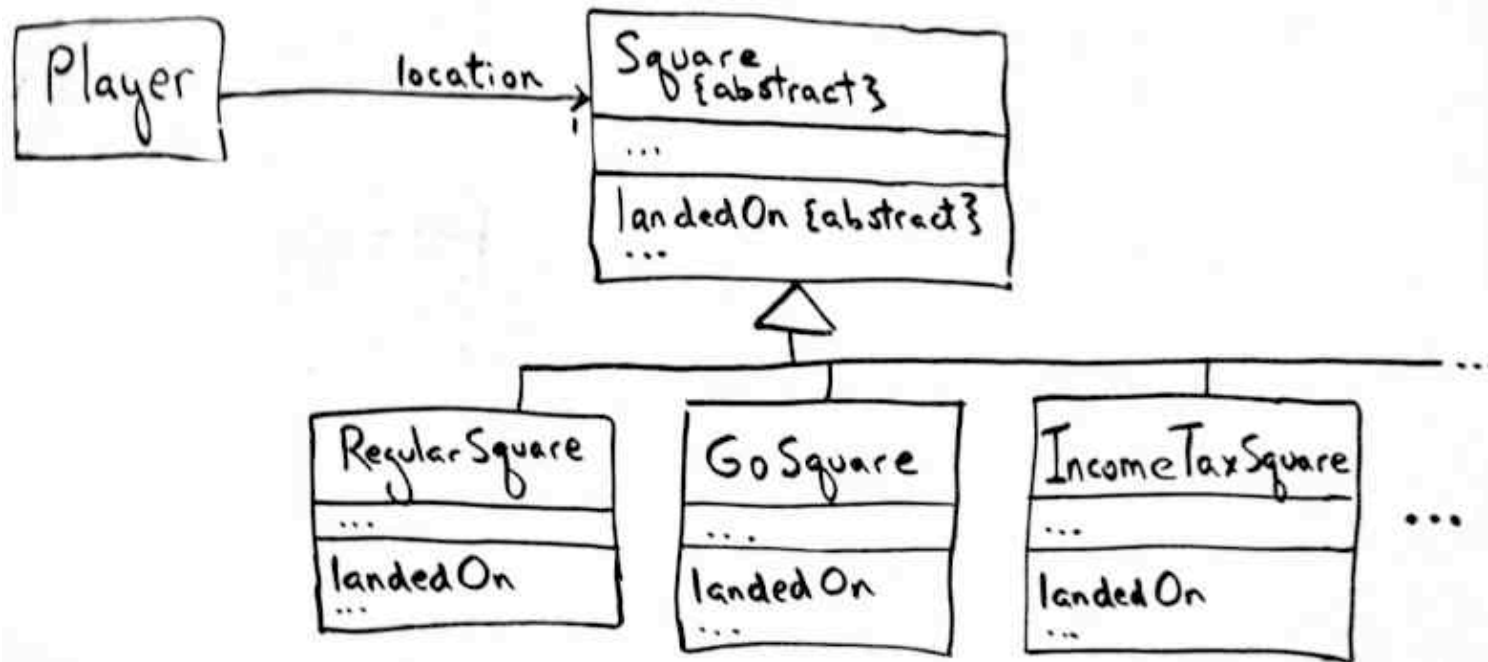
Esempio NextGen

- Vogliamo gestire i diversi sistemi di contabilità di terze parti. Dobbiamo far sì che il sistema di contabilità di POS NextGen possa comunicare con loro. Così facendo abbiamo più implementazioni simili ma che si adattano a differenti richieste



Esempio Monopoly

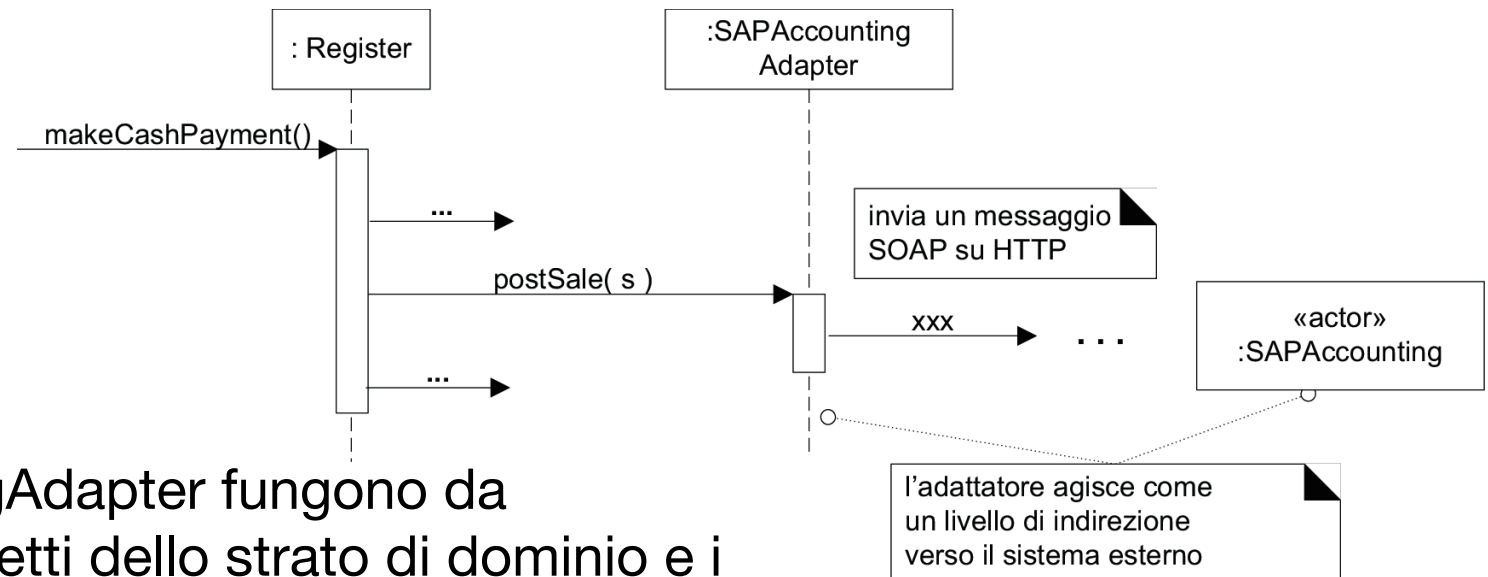
- Nel gioco del Monopoly le caselle possono avere un comportamento differente in base al loro tipo (se passo sul via avrò 200 dollari), anche se condividono delle caratteristiche comuni (posso raggiungere ogni casella)



Indirection

- **Problema:** dove assegnare una responsabilità, per evitare l'accoppiamento diretto tra più elementi? come disaccoppiare gli oggetti in modo da sostenere un alto riuso e un accoppiamento basso?
- **Soluzione:** si assegna la responsabilità ad un oggetto intermedio che media tra altri componenti o servizi, in modo che non ci sia un accoppiamento diretto tra di essi.
 - *L'intermediario crea una indirezione (indirection) tra le componenti.*
 - *Accoppiamento più basso tra componenti*

Esempio POS NextGen



- Gli oggetti `IAccountingAdapter` fungono da intermediari tra gli oggetti dello strato di dominio e i servizi esterni.
- Aggiungendo un livello di indicazione e il polimorfismo gli oggetti adattatori proteggono il progetto interno da variazioni nelle interfacce esterne

Protected Variations

- **Problema:** come progettare oggetti, sottosistemi e sistemi in modo tale che le variazioni o l'instabilità di questi elementi non abbiano si riversino su altri elementi?
- **Soluzione:** si identificano i punti in cui sono previste delle variazioni, poi si assegnano le responsabilità per creare una “interfaccia” stabile attorno a questi punti.

Protected Variations

Vantaggi:

- le estensioni richieste per nuove variazione sono facili da aggiungere
- è possibile introdurre nuove implementazione senza influire sui client
- favorisce Low Coupling
- è possibile ridurre l'impatto o il costo dei cambiamenti

Svantaggi:

- è inutile e costoso utilizzare PV per delle variazioni che non si verificheranno mai
- in alcuni momenti conviene di più riprogettare che realizzare la protezione da tutte le possibili variazioni